

intro to javascript

Credit Hours: 3-0

Course Outline

Introduction to javascript

16-Week Course Plan

Week 1

****Week 1: Introduction to JavaScript (3-0)****

Lecture 1.1: Introduction to JavaScript

Learning Objectives

- Define JavaScript and its role in web development
- Identify the key features of JavaScript

Concepts

JavaScript is a high-level, dynamic, and interpreted programming language. It is primarily used for client-side scripting on the web, allowing for interactive web pages and web applications. JavaScript is known for its flexibility and versatility, making it a popular choice for web development. Its key features include dynamic typing, first-class functions, and asynchronous programming.

Lecture 1.2: History and Evolution of JavaScript

Learning Objectives

- Understand the history and evolution of JavaScript
- Identify the key milestones in JavaScript's development

Concepts

JavaScript was first released in 1995 by Brendan Eich while working at Netscape Communications. Initially named "Mocha," it was later renamed to JavaScript due to its similarities to Java. The language has undergone significant changes over the years, with the release of ECMAScript in 1997, which standardized the language. Today, JavaScript is a widely used language for web development.

Lecture 1.3: Setting Up a JavaScript Environment

Learning Objectives

- Install a code editor or IDE for JavaScript development
- Set up a basic JavaScript project

Concepts

To start developing JavaScript applications, you need a code editor or Integrated Development Environment (IDE). Popular choices include Visual Studio Code, Sublime Text, and Atom. Once you have a code editor set up, you can create a new project by creating a new folder and adding a `script.js`` file to it. This will serve as the entry point for your JavaScript code.

Week 2

****Week 2: Functions and Control Flow****

****Distribution: 2-0****

Lecture 1: Introduction to Functions

Learning Objectives

- Define functions in JavaScript
- Understand the purpose of functions
- Learn basic function syntax

Concepts

Functions are reusable blocks of code that perform a specific task. They take arguments, execute code, and return values. Functions are defined using the `function` keyword followed by the function name, arguments in parentheses, and code inside curly brackets. For example:

```
```javascript
function greet(name) {
 console.log(`Hello, ${name}!`);
}
```
```

Functions can be called by their name, passing arguments as needed. This allows for code reuse and modularity.

Lecture 2: Control Flow Statements

Learning Objectives

- Understand the importance of control flow statements
- Learn basic syntax for if/else statements, for loops, and while loops

Concepts

Control flow statements determine the order in which code is executed. They allow for conditional execution, repetition, and iteration. The three main types of control flow statements are:

* ****If/else statements****: used for conditional execution

* ****For loops****: used for iteration over arrays or strings

* ****While loops****: used for repetition until a condition is met

Example:

```
```javascript
if (x > 5) {
 console.log("x is greater than 5");
}
```

```
} else {
 console.log("x is less than or equal to 5");
}
...
```

Control flow statements are essential for writing efficient and effective code.

## Week 3

**\*\*Week 3: Functions in JavaScript\*\***

### Lecture Title: Functions as First-Class Citizens

#### Learning Objectives:

- Understand the concept of functions as first-class citizens in JavaScript.
- Learn to declare and invoke functions.
- Understand function arguments and return values.

#### Concepts:

JavaScript functions are first-class citizens, meaning they can be passed as arguments to other functions, returned from functions, and stored in data structures. A function in JavaScript is a block of code that can be executed multiple times from different parts of the program.

To declare a function, use the `function` keyword followed by the function name and arguments in parentheses. The function body is enclosed in curly brackets `{}`.

```
```javascript  
function greet(name) {  
  console.log("Hello, " + name + "!");  
}  
...
```

Functions can be invoked by calling the function name followed by parentheses containing the arguments.

```
```javascript  
greet("John"); // Output: Hello, John!
...
```

Functions can also return values, which can be used in the calling code.

```
```javascript  
function add(a, b) {  
  return a + b;  
}
```

```
}
```

```
console.log(add(2, 3)); // Output: 5
```

```
...
```

Key Points:

- Functions are first-class citizens in JavaScript.
- Functions can be declared using the `function` keyword.
- Functions can be invoked by calling the function name with arguments.
- Functions can return values.

Lab:

Implement a function that takes a string as input and returns the string with all vowels removed.

Week 4

Week 4: Functions and Function Expressions (3-0)

Lecture 1: Functions in JavaScript

Learning Objectives

- * Understand the concept of functions in JavaScript
- * Learn how to declare and call functions
- * Understand the difference between function declarations and function expressions

Concepts

A function in JavaScript is a block of code that can be executed multiple times from different parts of the program. Functions can take arguments and return values. There are two ways to declare functions in JavaScript: function declarations and function expressions. Function declarations are declared using the `function` keyword, while function expressions are declared using the `function` keyword inside a variable assignment.

```
```javascript
```

```
function greet(name) {
 console.log("Hello, " + name);
}
```

```
greet("John"); // Output: Hello, John
```

```
...
```

### Lecture 2: Function Expressions

#### Learning Objectives

- \* Learn how to declare function expressions in JavaScript

- \* Understand the difference between function expressions and function declarations

### Concepts

Function expressions are declared using the `function` keyword inside a variable assignment. They can be assigned to a variable and called like any other function.

```
```javascript
const greet = function(name) {
  console.log("Hello, " + name);
};

greet("John"); // Output: Hello, John
```
```

## Lecture 3: Function Composition and Higher-Order Functions

### Learning Objectives

- \* Understand how to compose functions in JavaScript
- \* Learn about higher-order functions in JavaScript

### Concepts

Functions can be composed together to create new functions. Higher-order functions are functions that take other functions as arguments or return functions as output.

```
```javascript
function double(x) {
  return x * 2;
}

function increment(x) {
  return x + 1;
}

function compose(f, g) {
  return function(x) {
    return f(g(x));
  };
}

const doubleAndIncrement = compose(increment, double);

console.log(doubleAndIncrement(5)); // Output: 11
```
```

...

## **\*\*Assessment\*\***

- \* Quiz 4: Functions and Function Expressions (20 points)
- \* Assignment 4: Implement a function that takes a list of numbers and returns the sum of the even numbers (30 points)
- \* Reading: Chapter 4 of the textbook (50 points)

## **Week 5**

### **\*\*Week 5: Functions and Higher-Order Functions (3-0)\*\***

#### **Lecture 1: Introduction to Functions**

### **Functions in JavaScript**

In JavaScript, a function is a block of code that can be executed multiple times from different parts of a program. Functions allow us to write reusable code and make our programs more modular.

#### **Learning Objectives**

- Define a function in JavaScript
- Explain the benefits of using functions
- Identify common function parameters and return types

#### **Concepts**

- Function declaration
- Function expression
- Function parameters and return types

#### **Lecture 2: Function Types and Parameters**

### **Function Types and Parameters**

There are two main types of functions in JavaScript: built-in functions and user-defined functions. Functions can take parameters, which are values passed to the function when it's called.

#### **Learning Objectives**

- Distinguish between built-in and user-defined functions
- Explain the concept of function parameters
- Identify common function parameter types (number, string, object)

#### **Concepts**

- Built-in functions (Math, Date, String)
- User-defined functions

- Function parameters and data types

### Lecture 3: Higher-Order Functions

#### Higher-Order Functions

A higher-order function is a function that takes another function as an argument or returns a function as a result. Higher-order functions allow us to create more complex and reusable code.

#### Learning Objectives

- Define a higher-order function
- Explain the benefits of using higher-order functions
- Identify common higher-order function patterns (callback, map, filter)

#### Concepts

- Function as an argument
- Function as a return value
- Common higher-order function patterns (callback, map, filter)

#### Week 6

**\*\*Week 6: Functions and Scope (3-0)\*\***

#### Lecture 1: Functions

#### Learning Objectives

- \* Define functions in JavaScript
- \* Explain function declaration and function expression
- \* Describe the difference between function declaration and function expression

#### Concepts

Functions in JavaScript are blocks of code that can be executed multiple times from different parts of a program. A function declaration is a statement that begins with the `function` keyword, whereas a function expression is an expression that returns a function. Function expressions are often used as function arguments or as properties of objects.

```
```javascript
```

```
// Function declaration
```

```
function add(x, y) {  
  return x + y;  
}
```

```
// Function expression
```

```
const double = function(x) {
```



```
return x * 2;

}

...

```

A function can be defined using the `function` keyword followed by the function name and a list of parameters in parentheses. The function body is enclosed in curly brackets.

Lecture 2: Function Parameters and Return Values

Learning Objectives

- * Explain function parameters and their role in function calls
- * Describe how to return values from functions
- * Discuss the difference between `return` statement and implicit return

Concepts

Function parameters are the values passed to a function when it is called. These values are used to perform calculations or operations within the function. A function can return a value using the `return` statement. If no `return` statement is present, the function returns `undefined`.

```
````javascript
// Function with parameters
function greet(name, age) {
 console.log(`Hello, ${name}! You are ${age} years old.`);
}

// Function returning a value
function add(x, y) {
 return x + y;
}

...

```

## Lecture 3: Scope and Hoisting

### Learning Objectives

- \* Explain the concept of scope in JavaScript
- \* Describe how variables are scoped to the surrounding function or block
- \* Discuss hoisting and its implications on variable declaration

### Concepts

Scope in JavaScript refers to the region of the code where a variable is defined and accessible. Variables are scoped to the surrounding function or block. Hoisting is a mechanism that moves

variable declarations to the top of their scope, regardless of their actual position in the code.

```
```javascript
// Variable declared in the global scope
let globalVar = 'global';

// Variable declared in a function scope
function foo() {
  let localVar = 'local';
  console.log(localVar);
}
```
```

## Week 7

**\*\*Week 7: Functions in JavaScript\*\***

**\*\*Format: 3-0\*\***

### Lecture 1: Introduction to Functions

#### Learning Objectives

- Define a function in JavaScript
- Explain the purpose of functions in programming
- Identify the different types of functions in JavaScript

#### Concepts

Functions are reusable blocks of code that perform a specific task. In JavaScript, functions can be defined using the `function` keyword. Functions are used to organize code, reduce repetition, and improve code readability. There are two main types of functions in JavaScript: built-in functions and user-defined functions.

### Lecture 2: Function Parameters and Return Types

#### Learning Objectives

- Explain the concept of function parameters
- Identify the different types of function return types
- Define a function with parameters and return type

#### Concepts

Function parameters are variables that are passed to a function when it is called. They allow functions to accept input data and perform operations on it. Return types specify the data type that a function returns. In JavaScript, functions can return any data type, including primitive types and

objects.

## Lecture 3: Function Expressions and Arrow Functions

### Learning Objectives

- Explain the concept of function expressions
- Define an arrow function in JavaScript
- Identify the advantages of using arrow functions

### Concepts

Function expressions are functions that are defined using the `function` keyword, but are not assigned to a variable. Arrow functions are a concise way to define functions in JavaScript, using the `=>` operator. They are often used as event handlers and callbacks.

## Week 8

**\*\*Week 8: Function Abstraction and the `this` Keyword\*\***

### Lecture Title: Abstraction with Functions

#### Learning Objectives:

- Understand the concept of function abstraction
- Learn how to use functions to create abstractions
- Understand the `this` keyword in JavaScript

#### Concepts:

Functions in JavaScript are first-class citizens, meaning they can be passed around like any other variable. This allows for abstraction, a fundamental concept in programming. Abstraction is the process of hiding implementation details from the user, exposing only the necessary information to interact with the code.

```
```javascript
function greet(name) {
  console.log(`Hello, ${name}!`);
}
```
```

In the above code, the `greet` function abstracts away the implementation details of printing a greeting to the console. The user only needs to know that they can pass a name to the function to get a greeting.

#### Lecture Content:

Functions can also be used to abstract away complex logic. For example, a function can encapsulate a set of steps to perform a calculation.

```
```javascript
function calculateArea(width, height) {
  return width * height;
}
```
```

In the above code, the `calculateArea` function abstracts away the calculation logic, making it easier to reuse and understand.

#### **Lecture Content:**

The `this` keyword in JavaScript is a reference to the context in which a function is being called. In a function, `this` refers to the global object, which is the `window` object in a browser or the `global` object in a Node.js environment.

```
```javascript
console.log(this); // Output: window in a browser
```
```

#### **Lecture Content:**

However, in a function call, `this` refers to the object that the function is being called on.

```
```javascript
const person = {
  name: 'John',
  speak: function() {
    console.log(this.name);
  }
};

person.speak(); // Output: John
```
```

In the above code, the `speak` function is a method of the `person` object, so `this` refers to the `person` object.

#### **Lecture Content:**

In summary, functions are a powerful tool for abstraction in JavaScript, allowing us to hide implementation details and expose only the necessary information. Understanding the `this` keyword is crucial in JavaScript programming, as it affects how functions behave in different contexts.

#### **Lab Exercise:**

Implement a `Person` class with a `speak` method that uses the `this` keyword to access the person's name. Create an instance of the `Person` class and call the `speak` method to verify that the `this` keyword is working correctly.

---

## **Lecture Format: 2-1 (2 lectures, 1 lab)**

### **Week 9**

**\*\*Week 9: Functions (3-0)\*\***

#### **Lecture 1: Introduction to Functions**

##### **Learning Objectives**

- Define a function in JavaScript
- Explain the benefits of using functions
- Identify common use cases for functions

##### **Concepts**

In JavaScript, a function is a block of code that can be executed multiple times from different parts of a program. Functions are useful for reusing code, improving modularity, and reducing code duplication. A function typically takes in arguments, performs some operation, and returns a value.

#### **Lecture 2: Function Syntax and Parameters**

##### **Learning Objectives**

- Understand the basic syntax of a JavaScript function
- Identify the role of parameters in a function
- Explain the concept of default parameter values

##### **Concepts**

A JavaScript function is defined using the `function` keyword followed by the function name and a list of parameters enclosed in parentheses. Parameters are variables that are passed into a function when it is called. Functions can also have default parameter values, which are used if no value is provided when the function is called.

#### **Lecture 3: Function Return Types and Scope**

##### **Learning Objectives**

- Explain the concept of return types in functions
- Identify the scope of variables within a function
- Understand the concept of function hoisting

##### **Concepts**

A function in JavaScript can return a value of any data type, including numbers, strings, arrays, and objects. The scope of a variable within a function is limited to the function itself, unless the variable is declared using the `var` keyword with a global scope. Function hoisting is a JavaScript feature that allows functions to be accessed before they are declared.

## Week 10

**\*\*Week 10: Advanced JavaScript Topics (3-0)\*\***

### Lecture 1: Closures

#### Learning Objectives

- Understand the concept of closures in JavaScript
- Apply closures to create private variables and functions
- Use closures to create higher-order functions

#### Concepts

A closure is a function that has access to its outer scope, even when the outer function has returned. Closures are useful for creating private variables and functions, as well as higher-order functions. A closure is created when a function is returned from another function.

Example:

```
```javascript
function outer() {
  let count = 0;
  return function inner() {
    count++;
    return count;
  }
}

let counter = outer();
console.log(counter()); // 1
console.log(counter()); // 2
```
```

### Lecture 2: Immediately Invoked Function Expressions (IIFE)

#### Learning Objectives

- Understand the concept of IIFE in JavaScript
- Apply IIFE to create a scope for variables and functions

- Use IIFE to avoid global variables

### Concepts

An IIFE is a function that is executed immediately after it is defined. IIFE is used to create a scope for variables and functions, and to avoid global variables. IIFE is created by adding parentheses to a function expression.

Example:

```
```javascript
(function() {
  let x = 10;
  console.log(x); // 10
})();
```
```

## Lecture 3: JavaScript Modules

### Learning Objectives

- Understand the concept of JavaScript modules
- Apply JavaScript modules to organize code and avoid global variables
- Use JavaScript modules to export and import functions and variables

### Concepts

JavaScript modules are used to organize code and avoid global variables. A module is a file that exports functions and variables, which can be imported by other files. JavaScript modules are created using the `export` and `import` keywords.

Example:

```
```javascript
// module.js

export function add(x, y) {
  return x + y;
}

// main.js

import { add } from './module.js';

console.log(add(2, 3)); // 5
```
```

## Week 11

## **\*\*Week 11: Higher-Order Functions and Closures\*\***

**\*\*3-0\*\***

### **Lecture Title: Higher-Order Functions**

#### **Learning Objectives**

- \* Define higher-order functions and their purpose
- \* Explain the benefits of using higher-order functions
- \* Use map, filter, and reduce higher-order functions in JavaScript

#### **Concepts**

Higher-order functions are functions that take other functions as arguments or return functions as output. They are useful for abstracting away repetitive code and making it more modular. JavaScript provides several built-in higher-order functions, including map, filter, and reduce.

\* `map()`: Applies a given function to each element of an array and returns a new array with the results.

\* `filter()`: Creates a new array with all elements that pass the test implemented by the provided function.

\* `reduce()`: Applies a function against an accumulator and each element in the array (from left to right) to reduce it to a single output value.

### **Lecture Title: Closures**

#### **Learning Objectives**

- \* Define closures and their purpose
- \* Explain how closures work in JavaScript
- \* Use closures to create private variables and functions

#### **Concepts**

A closure is a function that has access to its outer function's scope, even when the outer function has returned. Closures are useful for creating private variables and functions that can be accessed only by the outer function.

\* A closure is created when a function is returned from another function.

\* The inner function has access to the outer function's scope, including its variables and functions.

\* Closures can be used to create private variables and functions that can be accessed only by the outer function.

### **Lecture Title: Advanced Topics in Higher-Order Functions and Closures**

#### **Learning Objectives**

- \* Discuss advanced topics in higher-order functions and closures



- \* Explain how to use higher-order functions and closures with async/await
- \* Use higher-order functions and closures to solve real-world problems

### Concepts

- \* Higher-order functions and closures can be used to create complex data processing pipelines.
- \* They can be used with async/await to create asynchronous data processing pipelines.
- \* Higher-order functions and closures can be used to create private state and behavior in objects.

## Week 12

**\*\*Week 12: Advanced JavaScript Topics\*\***

### Lecture Title

**\*\*Error Handling and Debugging\*\***

### Learning Objectives

- \* Understand the importance of error handling in JavaScript
- \* Learn how to use try-catch blocks to handle errors
- \* Understand the concept of debugging and how to use the console for debugging

### Concepts

In JavaScript, errors can occur due to various reasons such as syntax errors, runtime errors, or logical errors. Error handling is crucial to ensure that your code runs smoothly and doesn't crash unexpectedly. Try-catch blocks are used to handle errors in JavaScript. The try block contains the code that might throw an error, and the catch block contains the code that will be executed if an error occurs. The finally block is optional and is used to execute code regardless of whether an error occurred or not.

```
```javascript
try {
  // code that might throw an error
} catch (error) {
  // code to handle the error
} finally {
  // code to execute regardless of an error
}
```
```

The console object in JavaScript is used for debugging purposes. It allows you to log messages, inspect variables, and debug your code. You can use the console.log() method to log messages, and the console.error() method to log error messages.

```
```javascript
console.log('Hello World!');
console.error('Error occurred!');
```
```

### Lab

- \* Create a JavaScript file that throws an error using the throw statement.
- \* Use a try-catch block to handle the error and log the error message to the console.
- \* Use the console object to log a message to the console.

**\*\*Total Time: 3-0\*\***

## Week 13

**\*\*Week 13: Advanced JavaScript Topics (3-0)\*\***

### Lecture Title: Closures and Higher-Order Functions

#### Learning Objectives

- \* Understand the concept of closures in JavaScript
- \* Learn how to use higher-order functions for functional programming

#### Concepts

A closure is a function that has access to its outer scope, even when the outer function has returned. This allows the inner function to "remember" the variables of the outer function.

```
```javascript
function outer() {
  let x = 10;
  function inner() {
    console.log(x);
  }
  return inner;
}

let innerFunc = outer();
innerFunc(); // outputs 10
```
```

Higher-order functions are functions that take other functions as arguments or return functions as output. They are used for functional programming, which emphasizes immutability and the

avoidance of mutable state.

```
```javascript
function double(x) {
  return x * 2;
}

function triple(x) {
  return x * 3;
}

function apply(func, x) {
  return func(x);
}

console.log(apply(double, 5)); // outputs 10
console.log(apply(triple, 5)); // outputs 15
```
```

### Exercises

1. Create a closure that returns the sum of two numbers.
2. Write a higher-order function that takes a function and a number, and returns a new function that adds the number to the input of the original function.

## Week 14

**\*\*Week 14: Error Handling and Debugging (3-0)\*\***

### Lecture 37: Error Handling in JavaScript

#### Learning Objectives

- \* Understand the importance of error handling in JavaScript
- \* Learn how to use try-catch blocks to handle errors
- \* Identify common types of errors in JavaScript

#### Concepts

Error handling is a crucial aspect of JavaScript programming. It allows developers to anticipate and manage errors that may occur during the execution of their code. There are two primary types of errors in JavaScript: syntax errors and runtime errors. Syntax errors occur when the code is written incorrectly, while runtime errors occur when the code is executed. To handle errors, developers can use try-catch blocks. The try block contains the code that may throw an error, while the catch block contains the code that will execute if an error occurs. Common types of errors in JavaScript include type errors, range errors, and reference errors.

## Lecture 38: Debugging Techniques in JavaScript

### Learning Objectives

- \* Understand the importance of debugging in JavaScript
- \* Learn how to use the console object to debug JavaScript code
- \* Identify common debugging techniques in JavaScript

### Concepts

Debugging is the process of identifying and fixing errors in JavaScript code. The console object is a powerful tool for debugging JavaScript code. It allows developers to log messages, errors, and variables to the console, making it easier to identify and fix errors. Common debugging techniques in JavaScript include using the console object, using the debugger statement, and using the Chrome DevTools. By mastering these techniques, developers can write more efficient and effective JavaScript code.

## Week 15

**\*\*Week 15: Error Handling and Debugging\*\***

### Lecture Title: Error Handling in JavaScript

#### Learning Objectives:

- Understand the importance of error handling in JavaScript
- Learn how to use try-catch blocks to handle errors
- Understand the difference between synchronous and asynchronous errors

#### Concepts:

Error handling is crucial in JavaScript to prevent the program from crashing due to unexpected errors. JavaScript provides a built-in mechanism to handle errors using try-catch blocks. The try block contains the code that might throw an error, while the catch block contains the code that will be executed if an error occurs.

```
```javascript
try {
  // code that might throw an error
} catch (error) {
  // code to handle the error
}
```
```

Synchronous errors occur when an error is thrown and not caught, causing the program to terminate. Asynchronous errors occur when an error is thrown in a callback function, and the program continues to execute without crashing.

## **Lecture Title: Debugging Techniques in JavaScript**

### **Learning Objectives:**

- Understand the importance of debugging in JavaScript
- Learn how to use console.log to debug JavaScript code
- Understand how to use the browser's developer tools to debug JavaScript code

### **Concepts:**

Debugging is an essential step in the development process to identify and fix errors in the code. JavaScript provides several tools to debug code, including console.log and the browser's developer tools. Console.log can be used to print variables and expressions to the console, while the browser's developer tools provide a more comprehensive debugging environment.

```
```javascript
console.log("Hello, World!");
```
```

The browser's developer tools can be used to set breakpoints, inspect variables, and step through code line by line.

## **Lecture Title: Best Practices for Error Handling and Debugging**

### **Learning Objectives:**

- Understand the best practices for error handling in JavaScript
- Learn how to write robust and error-free code
- Understand how to use error handling and debugging techniques in real-world applications

### **Concepts:**

Error handling and debugging are crucial skills for any JavaScript developer. Best practices for error handling include using try-catch blocks, logging errors, and providing informative error messages. Debugging techniques include using console.log, setting breakpoints, and inspecting variables.

By following these best practices, developers can write robust and error-free code that is maintainable and efficient.

## **Week 16**

**\*\*Week 16: Advanced JavaScript Topics (3-0)\*\***

### **Lecture 1: Asynchronous Programming with Callbacks**

#### **Learning Objectives:**

- Understand the concept of asynchronous programming
- Learn how to use callbacks in JavaScript

**Concepts:**

Asynchronous programming allows for non-blocking execution of code. Callbacks are functions passed as arguments to other functions, to be executed later. In JavaScript, callbacks are used to handle asynchronous operations such as reading files, making HTTP requests, or interacting with databases. A callback function is typically called with an argument, which is used to pass data back to the calling function.

## Lecture 2: Asynchronous Programming with Promises

**Learning Objectives:**

- Understand the concept of promises in JavaScript
- Learn how to use promises to handle asynchronous operations

**Concepts:**

Promises are a more modern and elegant way to handle asynchronous operations in JavaScript. A promise represents a value that may not be available yet, but will be resolved at some point in the future. When a promise is resolved, its associated value is returned, and when it's rejected, an error is thrown. Promises can be chained together to handle complex asynchronous operations.

## Lecture 3: Asynchronous Programming with Async/Await

**Learning Objectives:**

- Understand the concept of async/await in JavaScript
- Learn how to use async/await to write asynchronous code that's easier to read and maintain

**Concepts:**

Async/await is a syntax sugar on top of promises, allowing for cleaner and more readable asynchronous code. When an async function is called, it returns a promise, and the await keyword is used to pause execution until the promise is resolved. This makes asynchronous code look and feel synchronous, making it easier to write and maintain.